Manufacturing Operational Insights Report Chat with Logistics Documents (RAG)

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	HATIM BOHRA	EN22CS301418
2	INDRA KAMAL PATEL	EN22CS301434
3	GOUTAM JAT	EN22CS301371
4	ISHIKA AGRAWAL	EN22CS301442
5	HITESH AHUJA	EN22CS301426
6	HARSH SOLANKI	EN22CS301394

Group Name: Group 09D4

Project Number: GAI-45

Industry Mentor Name: Mr. Suraj Nayak

University Mentor Name: Prof. Divya Kumawat

Academic Year: 2025-26

Problem Statement & Objectives

1. Problem Statement: Chat with Logistics Documents (RAG)
2. Project Objectives: To build a chatbot that understands user queries in natural language and retrieves relevant information from logistics data or documents using *Retrieval-Augmented Generation (RAG)* techniques.
3. Scope of the Project: he scope of the Logistics ChatBot project includes the development of an AI-powered assistant capable of understanding and responding to logistics-related queries using Retrieval-Augmented Generation (RAG).

Proposed Solution

1. Key features

- AI-powered Logistics ChatBot
- Retrieval-Augmented Generation (RAG) based architecture
- Semantic search using vector embeddings
- Context-aware response generation
- Natural Language Query Processing
- Modular and scalable design
- Easy integration with web interface

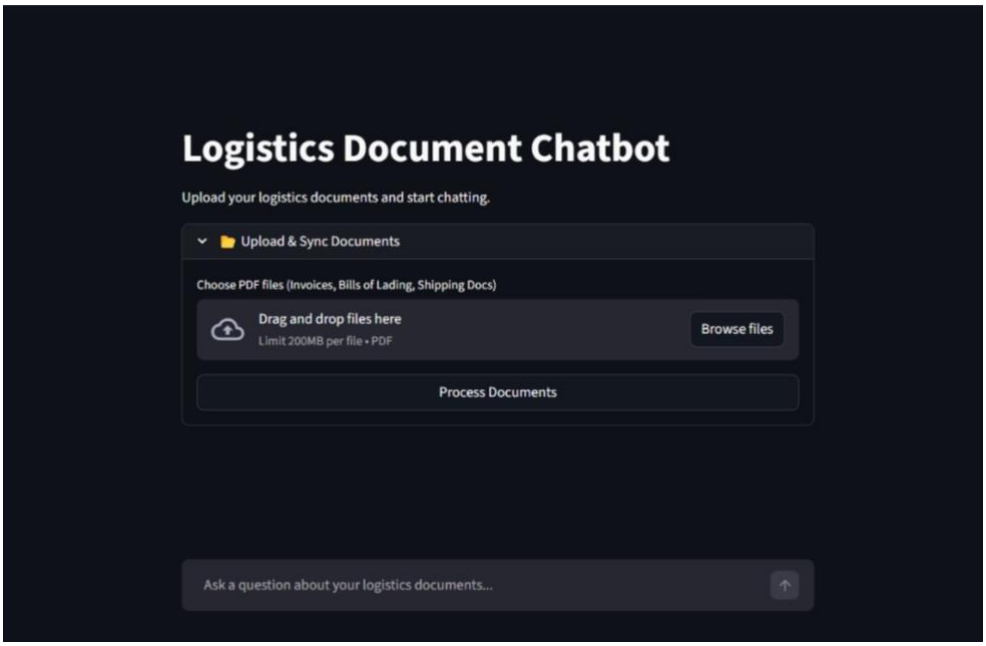
2. Overall Architecture / Workflow

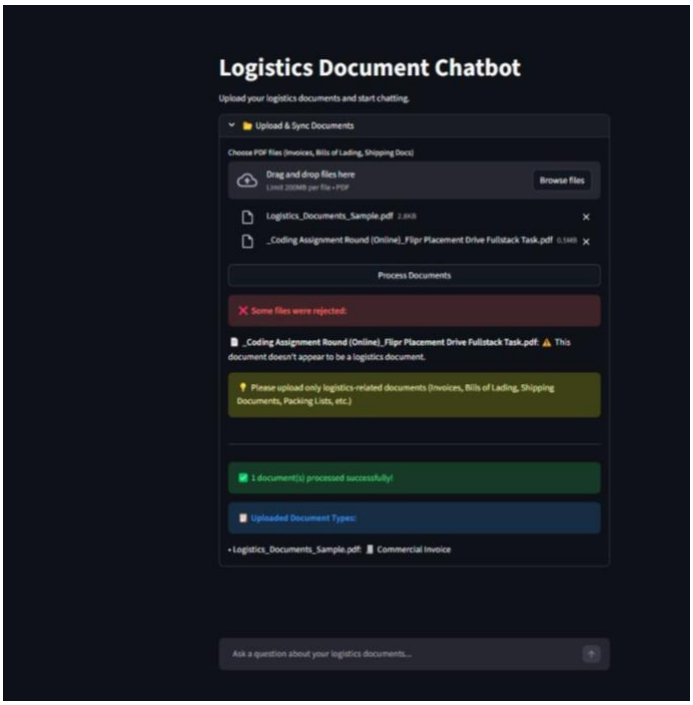
Step 1: User enters query through application interface (`app.py`)
Step 2: Query is forwarded to the RAG Engine (`rag_engine.py`)
Step 3: Query is converted into vector embeddings
Step 4: Semantic search retrieves relevant document chunks from vector store
Step 5: Retrieved context + User query sent to LLM
Step 6: LLM generates accurate logistics-related response
Step 7: Response displayed back to user
Workflow Summary:
User → Application Layer → RAG Engine → Vector Store Retrieval → LLM → Response

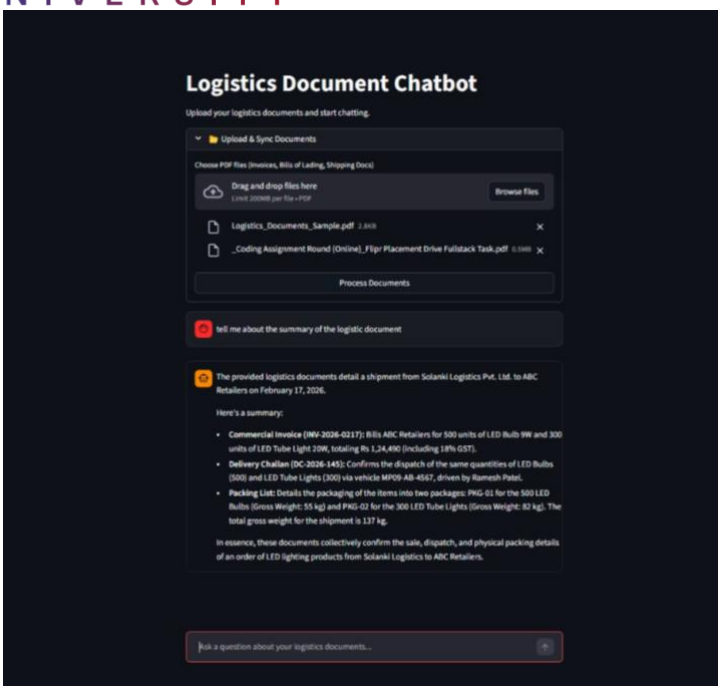
3. Tools & Technologies Used

- **Python** – Core programming language
- **LangChain** – RAG pipeline management
- **OpenAI API** – Large Language Model integration
- **Vector Database (Embeddings)** – Semantic search
- **Flask / Streamlit** – Web application interface
- **GitHub** – Version control and project hosting

Results & Output







Key Outcomes

- Successfully developed a **RAG-based Logistics ChatBot**
- Implemented semantic search using vector embeddings
- Achieved context-aware and accurate response generation
- Reduced dependency on manual query handling
- Improved response efficiency for logistics-related queries
- Built a modular and scalable architecture
- Demonstrated practical implementation of LLM + Retrieval system
- Enhanced understanding of real-world AI application development

Conclusion

The Logistics ChatBot project successfully demonstrates the implementation of a **Retrieval-Augmented Generation (RAG)** based AI system for handling logistics-related queries. By integrating semantic search with a Large Language Model, the system provides accurate, context-aware responses instead of generic answers.

The project combines backend logic (rag_engine.py) and application interface (app.py) to create a complete working solution.

Key Learnings:

- Practical implementation of RAG architecture
- Working with vector embeddings and semantic search
- Integration of LLMs using APIs
- Building modular and scalable AI applications
- Understanding real-world AI deployment workflow

Future Scope & Enhancements

- Integration with real-time logistics databases
- Deployment on cloud platforms (AWS / Azure / GCP)
- Adding multi-language support
- Improving UI/UX for better user interaction
- Implementing authentication and user management
- Adding chat history and analytics dashboard
- Fine-tuning models for domain-specific optimization
- Integration with enterprise ERP or supply chain systems